

第23回 必ず書かせる:抽象クラス

今回のゴール

- `abstract` を付けたクラスは `new` できないことを説明できる。
- 抽象メソッドを使って、「 subclasses で必ず書くメソッド」を決められる。
- 抽象メソッドを書き忘れたときのエラーを読み、修正できる。
- 第16回～第22回の内容を確認し、自分の苦手な部分を知る。

前回とのつながり

第22回で、勇者・魔法使い・戦士を 1 つの `Character[]` にまとめ、全員に `attack` を命令できるようになった。しかし、今の `Character` には次の問題がある。

- `new Character("謎の人", 10, 1)` のように、職業のない「ただのキャラクター」も作れてしまう。
- 新しい職業のクラスを作ったとき、`attack` を書き忘れても、エラーにならずに `Character` のふつうの攻撃が使われる。

今回は、この 2 つを 抽象クラス で防ぐ。

実習時のフォルダ構成

```
ワークスペース
├─ java23_sample
│   └─ src
│       ├─ Character.java
│       ├─ Hero.java
│       ├─ Wizard.java
│       ├─ Warrior.java
│       ├─ Enemy.java
│       └─ Main.java
└─ java23_exercise
    └─ src
        ├─ (sample と同じ 6 ファイル)
        └─ Priest.java ← 演習2で自分で作る
```

- 配布ファイル: `プログラム/第23回/sample/`、`プログラム/第23回/exercise/`
- 解答例: `プログラム/第23回/answer/`
- `Warrior.java` は第22回の演習で作ったクラス。

今日の時間の目安

時間	内容
30 分	講義:抽象クラス・エラー体験
15 分	前半の確認(第16回～第22回)
35 分	演習
10 分	確認問題・振り返り

講義内容

1. abstract:new できないクラス

プログラム/第23回/sample/Character.java (変わったところ)

```
// 抽象クラス:Character そのものは new できない。必ず Hero や Enemy などの子クラスを作って使う
public abstract class Character {
    ...

    // 抽象メソッド:攻撃のしかたはキャラクターごとに違うので、子クラスで必ず書く
    public abstract void attack(Character target, int turn);

    // ふつうの攻撃 (子クラスの attack の中から使える)
    public void normalAttack(Character target) {
        System.out.println(name + "の攻撃! " + target.getName() + "に" + power + "のダメージ");
        target.damage(power);
    }

    ...
}
```

書き方	意味
<code>public abstract class Character</code>	抽象クラス。 <code>new Character(...)</code> ができない
<code>public abstract void attack(...);</code>	抽象メソッド。中身(<code>{ }</code>)を書かず、 <code>;</code> で終わる。子クラスで必ずオーバーライドする

- 抽象クラスにも、ふつうのフィールド・コンストラクタ・メソッドを書ける。HP の管理などの共通部分は、今までどおり `Character` に残る。
- `Character` 型の変数や配列は、今までどおり使える(`Character[] party` など)。作れないのは `Character` そのもののインスタンスだけ。

2. ふつうの攻撃は normalAttack に

第21回・第22回では、会心の一撃でないターンに `super.attack(...)` で親クラスの攻撃を使っていた。`attack` が抽象メソッドになって中身がなくなったので、ふつうの攻撃を `normalAttack` という別の名前で `Character` に残した。

プログラム/第23回/sample/Hero.java (変わったところ)

```
@Override
public void attack(Character target, int turn) {
    if (turn % 3 == 0) {
        ...
    } else {
        normalAttack(target);    // Character のふつうの攻撃
    }
}
```

`Enemy.java` も同じように、強攻撃でないターンは `normalAttack(target)` を呼ぶ。

3. 実行する前に予想する

`Main.java` では、敵が強いトロル(HP 110、攻撃力 12)に変わった。1、1、1、2、1、1 の順に入力する。

- ・ 4 ターン目のトロルの攻撃はどうなるか(`Enemy` の `attack` を読む)。
- ・ たおれたメンバーは、次のターンに攻撃するか(`Main` の `case 1` を読む)。

4. 実行結果(4 ターン目から最後まで)

```
--- ターン4 ---
勇者 HP:18/30
魔法使い HP:8/20
戦士 HP:28/40
トロル HP:22/110
1: 攻撃
2: 薬草を使う
3: にげる
4: ためる
コマンドを選ぶ > 2
勇者は薬草を使った！ HPが30/30になった
トロルの強攻撃！ 魔法使いに24のダメージ

--- ターン5 ---
勇者 HP:30/30
魔法使い HP:0/20
戦士 HP:28/40
トロル HP:22/110
1: 攻撃
2: 薬草を使う
3: にげる
4: ためる
コマンドを選ぶ > 1
勇者の攻撃！ トロルに8のダメージ
戦士のおのの一撃！ トロルに10のダメージ
```

トロルの攻撃！ 戦士に12のダメージ

--- ターン6 ---

勇者 HP:30/30

魔法使い HP:0/20

戦士 HP:16/40

トロル HP:4/110

1: 攻撃

2: 薬草を使う

3: にげる

4: ためる

コマンドを選ぶ > 1

会心の一撃！

勇者の攻撃！ トロルに12のダメージ

トロルをたおした！ 経験値60を手に入れた

- 抽象クラスにしても、バトルの動きは第22回と変わらない。変わったのは「作れるもの」と「書き忘れたときにどうなるか」。

5. エラーを体験する

1. `Main.java` の `main` の最初に、`Character someone = new Character("謎の人", 10, 1);` と書いて保存する。
 2. `Character` は抽象クラスなのでインスタンスを作れない、というエラーになることを確認する。確認したら消す。
 3. `Wizard.java` の `attack` メソッド(`@Override` から `}` まで)を、コメントにして保存する(範囲を選んで `Ctrl + /`)。
 4. `Wizard` が抽象メソッド `attack` を実装していない、というエラーになることを確認する。Pleiades では、エラーの印をクリックすると「実装されていないメソッドの追加」という修正候補が出ることもある。
 5. コメントを外して元に戻す。
- 第22回までは、`attack` を書き忘れてもエラーにならず、気付かないまま「ふつうの攻撃」になっていた。抽象メソッドにすると、書き忘れをコンパイラが見つけてくれる。

前半の確認(15分)

第16回～第22回で学んだことの確認。何も見ずに解く。

問1 出力は何か。

```
int x = 17;
System.out.println(x / 5 + " " + x % 5);
```

問2 出力は何か。

```
String[] s = { "A", "B", "C" };
for (int i = s.length - 1; i >= 0; i--) {
    System.out.print(s[i]);
}
```

問3 出力は何か。

```
public static void main(String[] args) {
    System.out.println(twice(twice(3)));
}

static int twice(int n) {
    return n * 2;
}
```

問4 出力は何か(クラスはサンプルと同じ)。

```
Character a = new Hero("勇者", 30, 8, 2);
Character b = a;
b.damage(10);
System.out.println(a.getHp());
```

問5 Main クラスで `hero.hp = 5;` と書いた。コンパイルできるか。理由も答える。

問6 出力は何か(クラスはサンプルと同じ)。

```
Character c = new Warrior("戦士", 40, 10);
c.attack(new Warrior("的", 50, 1), 1);
```

■ 答え

- 問1: 3 2 (第16回:整数の割り算と余り)
- 問2: CBA (第17回:配列と `for`。最後の添字は `length - 1`)
- 問3: 12 (第18回:内側の `twice(3)` が先に 6 を返し、`twice(6)` が 12 を返す)
- 問4: 20 (第19回: `a` と `b` は同じインスタンスを指す)
- 問5: できない。 `hp` は `private` なので、`Character` の外から使えない(第20回)
- 問6: 戦士のおのの一撃! 的に10のダメージ (第21回・第22回:動くのは中身の `Warrior` の `attack`)

間違えた問題の回のレジメを、次回までに読み直す。

演習1 サンプルを実行する

`java23_sample` でサンプルを実行し、予想と結果を比べる。

演習2 僧侶を仲間にする

`java23_exercise` を作り、`プログラム/第23回/exercise/` の 6 ファイルを入れる。

演習2-1 Priest クラスを作る(エラーから始める)

1. 新しいクラス `Priest` (僧侶)を作る。`Character` を受けつぎ、コンストラクタは名前・HP・攻撃力を受け取って `super(...)` に渡す。まだ `attack` は書かない。
2. 保存して、どんなエラーが出るか読む。エラーメッセージの中の、実装が必要なメソッドの名前を確認する。
3. `attack` をオーバーライドし、`僧侶は杖でたたいた！ トロルに4のダメージ` と表示してダメージを与える。

演習2-2 パーティーに加える

- `Main` で僧侶(HP 25、攻撃力 4)を作り、配列 `party` の最後に加える。

完成条件(1 ターン目):

```
--- ターン1 ---
勇者 HP:30/30
魔法使い HP:20/20
戦士 HP:40/40
僧侶 HP:25/25
トロル HP:110/110
1: 攻撃
2: 薬草を使う
3: にげる
4: ためる
コマンドを選ぶ > 1
勇者の攻撃！ トロルに8のダメージ
魔法使いは火の魔法をとこなえた！ トロルに10のダメージ
戦士のおのの一撃！ トロルに10のダメージ
僧侶は杖でたたいた！ トロルに4のダメージ
トロルの攻撃！ 魔法使いに12のダメージ
```

- 僧侶は攻撃が弱い。「回復できる」という僧侶らしい力は、第28回で追加する。

発展(任意)

- `Character` に抽象メソッド `public abstract String getJob();` を追加し、各職業のクラスで職業名(`"勇者"`、`"魔法使い"` など)を返すようにする。`showStatus` で `[勇者] 勇者 HP:30/30` のように表示する。`Enemy` にも必要になることに注意する。

確認問題

問1 コンパイルできるか。

```
abstract class Monster {  
    abstract void cry();  
}
```

```
Monster m = new Monster();
```

問2 出力は何か。

```
abstract class Monster {  
    void hello() {  
        System.out.println("やあ");  
    }  
  
    abstract void cry();  
}  
  
class Slime extends Monster {  
    @Override  
    void cry() {  
        System.out.println("ぶる");  
    }  
}
```

```
Monster m = new Slime();  
m.hello();  
m.cry();
```

問3 問2の `Monster` を使って、次のクラスを作った。コンパイルできるか。

```
class Bat extends Monster {  
}
```

■ 答え

- 問1: できない(抽象クラスはインスタンスを作れない)
- 問2: やあ と ぶる (抽象クラスにもふつうのメソッドを書ける。`Monster` 型の変数に子クラスのインスタンスを入れられる)
- 問3: できない(抽象メソッド `cry` を実装していない)

振り返り

- `Character` を抽象クラスにして、何ができなくなったか。それがなぜ良いのか説明できるか。
- 抽象メソッドを書き忘れたときのエラーを読んで直せたか。

- 前半の確認で間違えた回はどこか。

次回へのつながり

今のパーティーは、最初に作った配列のまま人数が変わらない。敵も 1 体だけ。次回は、戦闘中に仲間が加わったり、たおした敵がいなくなったりするように、人数が変わる「リスト」を使う。